

Error Handling in Reflection

Document #: P3560R0 [[Latest](#)] [[Status](#)]
Date: 2025-01-12
Project: Programming Language C++
Audience: EWG, LEWG
Reply-to: Peter Dimov
<pdimov@gmail.com>
Barry Revzin
<barry.revzin@gmail.com>

Contents

- [1 Introduction](#)
- [2 Exception Type](#)
 - [2.1 Encoding](#)
 - [2.2 Single or Multiple Types](#)
- [3 Recoverable or Unrecoverable](#)
- [4 Proposed Wording](#)
 - [4.1 \[meta.reflection.synop\]](#)
 - [4.2 \[meta.reflection.exception\]](#)
 - [4.3 \[meta.reflection.operators\]](#)
 - [4.4 \[meta.reflection.names\]](#)
 - [4.5 \[meta.reflection.queries\]](#)
 - [4.6 \[meta.reflection.member.queries\]](#)
 - [4.7 \[meta.reflection.layout\]](#)
 - [4.8 \[meta.reflection.extract\]](#)
 - [4.9 \[meta.reflection.substitute\]](#)
 - [4.10 \[meta.reflection.result\]](#)
 - [4.11 \[meta.reflection.define.aggregate\]](#)
- [5 References](#)

§ 1 Introduction

In [\[P2996R8\]_\(Reflection for C++26\)](#), we had to answer the question of what the error handling mechanism should be. We considered four options:

1. Returning an invalid reflection (similar to NaN for floating point)
2. Returning a `std::expected<T, E>` for some reflection-specific error type E
3. Failing to be a constant expression
4. Throwing an exception of type E, for some type E.

Option (1) doesn't work well, because not all reflection functions return `std::meta::info`. Some (such as `members_of`) return `vector<info>`, some (such as `identifier_of`) return `string_view`, and `extract<T>` even returns `T`. A NaN reflection doesn't solve the problem.

Option (2) places a heavy syntactic burden on user code, because `std::expected` needs to be unwrapped manually, without help from the language.

Option (3) doesn't provide any means for user code to recover from an error.

At the time we had to make the decision, option (4) was essentially equivalent to (3), because throwing an exception wasn't a constant expression, so we settled on option (3). However, since the adoption of [\[P3068R6\] \(Allowing exception throwing in constant-evaluation\)](#), that has changed, and option (4) has become viable.

Using exceptions to signal errors doesn't suffer from the problem with option (1), because it's a strategy that can be used regardless of the return type. It also doesn't require syntactic changes to the user code.

Ordinarily, for runtime functions, exception handling might be avoided for reasons of binary size and runtime overhead; it also imposes the requirement that the API can't be used with exceptions disabled (which is nonstandard, but nevertheless highly popular.)

However, none of these objections apply to exceptions used at compile time. They have no binary footprint, don't affect the run time, and there is no reason for a compiler to not allow them even in "no exceptions" mode (because they are entirely contained to program compilation.)

Therefore, we believe that we need to adopt option (4) as the error handling strategy for reflection functions.

§ 2 Exception Type

To signal errors via throwing an exception, we need to settle on an exception type (or types) which to throw.

Since these exceptions will never escape to runtime, we don't need to be concerned with deriving their type(s) from `std::exception`. However, it would be desirable for the exceptions to carry enough information for error recovery (when caught), enough information for high quality error messages (when uncaught), and for them to be suitable for error handling in user `constexpr` and `constexpr` functions as well, in addition to standard ones.

To that end, we propose the following exception type:

```
namespace std::meta {  
  
class exception  
{  
public:  
    constexpr exception(u8string_view what,  
                        info from,
```

```

        source_location where =
        source_location::current());

constexpr u8string_view what() const;
constexpr info from() const;
constexpr source_location where() const;
};

}

```

`exception::what()` is a string describing the error; `exception::from()` is a reflection of the function (or function template) from a call to which the error originated; and `exception::where()` is the source location of the call to that function.

For example, the following function

```

constexpr auto f()
{
    return members_of(^int);
}

```

will throw an exception of type `std::meta::exception` for which `what()` will return (for example) `u8"invalid reflection operand"`, `from()` will return `^std::meta::members_of`, and `where()` will return a `std::source_location` object pointing at the call to `members_of` inside `f`.

Suppose a user wishes to write a `constexpr` function that only accepts class type reflections. It would be possible to use `std::meta::exception` to signal errors as follows:

```

constexpr auto user_fn(info type, source_location where =
    source_location::current())
{
    if( !is_class_type(type) )
    {
        throw std::meta::exception(u8"not a class type", ^user_fn,
            where);
    }

    // carry on
}

```

§ 2.1 Encoding

What encoding should we use for the string describing the error, and what character type?

The encoding is left unspecified in the runtime case (`std::exception::what()`), which is generally regarded as a defect ([\[LWG4087\]](#)). Since we are designing a new component, we should not repeat that mistake, and specify the encoding of `meta::exception::what()`.

Since the string describing the error can be constructed from components coming from multiple sources, it should use an encoding that can represent any of these substrings. That is, it should use UTF-8.

The principled way to reflect this fact in the type system is to use `u8string_view`. However, there are strong, purely pragmatic, arguments in favor of using `string_view` instead.

`char8_t` has nearly zero support in the standard library, which makes it very inconvenient to use. Suppose, for example, that we are writing a function `member_of(info x, size_t i)` that returns `members_of(x)[i]`:

```
constexpr info member_of(info x, size_t i, source_location where =
    source_location::current())
{
    auto v = members_of(x);

    if( i >= v.size() )
    {
        throw meta::exception( u8"invalid member index", ^^member_of,
            where );
    }

    return v[i];
}
```

Further suppose that we want to provide a more descriptive error string, e.g. `"152 is not a valid member index"`, where 152 is the value of `i`.

There's basically no way to easily do that today. We can't use `std::format` to create a `u8string`, there is no `std::to_u8string`, there is even no equivalent of `to_chars` that would produce a `char8_t` sequence.

In contrast, if the constructor took `std::string_view`, we could have used any of these.

So maybe we should just use `string_view`? But that's not consistent with the current state of [\[P2996R8\]](#). It did start out using `string_view` everywhere, and had to be rewritten to supply additional `u8string_view` interfaces, for good reasons.

Consider, for instance, `std::meta::identifier_of(x)`. It can fail for two reasons: if the entity to which `x` refers has no associated identifier, or if it does, but that identifier is not representable in the literal encoding.

We are changing these failures from hard errors (not a constant expression) to throwing `meta::exception`. A sketch implementation of `identifier_of`, then, would look like this:

```
constexpr string_view identifier_of(info x)
{
    if( !has_identifier(x) )
    {
        throw meta::exception(u8"entity has no identifier",
            ^^identifier_of, ...);
    }
}
```

```

}

auto id = u8identifier_of(x);

if( !is_representable(id) )
{
    throw meta::exception(u8"identifier '"s + id + u8"'is not
        representable", ^^identifier_of, ...);
}

// convert id to the literal encoding and return it
}

```

For quality of implementation reasons, we want to include the identifier in the error description string we pass to the exception constructor, so that the subsequent error message will say "the identifier 'риба' is not representable" and not just "identifier not representable".

There is no way to do that if we take and return `string_view` from the exception constructor and `what()`. Since the failure is caused by the identifier not being representable in the literal encoding, it trivially follows that we can't put it into an error string that uses the literal encoding.

That is why we believe that the currently proposed interface, taking and returning `u8string_view`, is the path to take in order to maintain consistency with the current design of [\[P2996R8\]](#), which is the result of extensive discussions in SG16.

(The fact that the standard library doesn't provide adequate support for `char8_t` strings is fixable by... adding that support.)

What if, however, we follow [\[P2996R8\]](#) even more closely and provide two constructors for `meta::exception`, one taking `u8string_view` and one taking `string_view`?

This is possible, and will allow users to take advantage of present standard library facilities for string manipulation when constructing the exception. We don't propose it here for two reasons. One, it allows writing code that isn't quite correct, even though it would probably work on all tested platforms.

Consider this hypothetical user function:

```

constexpr auto user_fn(info x)
{
    // ...

    info y = member_of(x, i);

    if( is_private(y) )
    {
        throw meta::exception(std::format("member {} is private",
            identifier_of(y)), ^^user_fn, ...);
    }
}

```

```
    // ...  
}
```

If the identifier of `y` is not representable in the literal encoding, this function will throw an exception that says that the identifier is not representable, instead of the correct one saying that the member is private.

Since in the common case the literal encoding is UTF-8, this mistake is going to go undetected.

Whether this is a problem worth worrying about is up for debate. The principled approach is to only take `u8string_view`, forcing correct user code whether the user likes it or not. A less principled but considerably more pragmatic approach would prefer ease of use over forced (over-)correctness and add the `string_view` constructor.

But the second reason for our not proposing the additional `string_view` constructor is that it's easy to add at a later date.

§ 2.2 Single or Multiple Types

We are proposing a single exception type. The runtime analogy is `std::system_error` as opposed to a hierarchy of exception types.

This in principle makes user code that wishes to inspect the failure reason and do different things depending on it less convenient to write. It would have to look like this

```
catch( meta::exception const& x )  
{  
    if( x.from() == ^^identifier_of )  
    {  
        // handle errors originating from identifier_of  
    }  
    else if( x.from() == ^^members_of )  
    {  
        // handle errors originating from members_of  
    }  
    // ...  
}
```

instead of, hypothetically, like this

```
catch( meta::identifier_exception const& x )  
{  
    // handle errors originating from identifier_of  
}  
catch( meta::members_exception const& x )  
{  
    // handle errors originating from members_of  
}
```

```
}  
// ...
```

(exception type names are illustrative.)

We don't propose an exception hierarchy here. Designing a proper exception hierarchy is not something we can realistically do in the C++26 timeframe. It's not as straightforward as just using an exception per function because functions can fail for multiple reasons, and client code may well wish to distinguish between these.

Furthermore, an exception hierarchy can be designed at a later date, with the functions changed to throw an appropriate type derived from the currently proposed `meta::exception`. Code written against this proposal will continue to work unmodified, and new code would be able to use more specific catch clauses.

§ 3 Recoverable or Unrecoverable

We went through the proposed API in [\[P2996R8\]](#) and we think that all of the library functions should be recoverable — that is failing to meet the requirements of the function should be an exception rather than constant evaluation failure — with a single exception, `std::meta::define_aggregate`.

`define_aggregate` isn't likely to be used from a context from which recovery is meaningful, and even if it were, for meaningful recovery we would have to guarantee that the partial effects of a failure have been rolled back (as a definition containing some of the members may already have been produced at the point where the error is detected.) We don't believe that imposing this requirement is warranted or worth the cost.

The rest of the library functions are straightforwardly fallible, so the ability to recover from them is desirable.

§ 4 Proposed Wording

The wording here introduces a new type `std::meta::exception` and defines it.

Otherwise it's pretty rote changing all the error handling from something of the form “*Constant When: C*” to “*Throws: meta::exception unless C*”.

§ 4.1 [meta.reflection.synop]

Add to the synopsis in [meta.reflection.synop:]

```
namespace std::meta {  
    using info = decltype(^^::);  
  
    + // [meta.reflection.exception], class exception  
    + class exception;
```

```
// ...  
}
```

§ 4.2 [meta.reflection.exception]

Add a new subclause as follows:

Class exception, [meta.reflection.exception]

```
class exception  
{  
private:  
    u8string what_;           // exposition only  
    info from_;              // exposition only  
    source_location where_; // exposition only  
  
public:  
    consteval exception(u8string_view what, info from,  
        source_location where = source_location::current()) noexcept;  
  
    exception(exception const&) = default;  
    exception(exception&&) = default;  
  
    exception& operator=(exception const&) = default;  
    exception& operator=(exception&&) = default;  
  
    consteval u8string_view what() const noexcept;  
    consteval info from() const noexcept;  
    consteval source_location where() const noexcept;  
};
```

1 Reflection functions throw exceptions of type `std::meta::exception` to signal an error. `std::meta::exception` is a `constexpr`-only type.

```
    consteval exception(u8string_view what, info from,  
        source_location where = source_location::current()) noexcept;
```

2 *Effects:* Initializes *what_* with *what*, *from_* with *from* and *where_* with *where*.

```
    consteval u8string_view what() const noexcept;
```

3 *Returns:* *what_*.

```
    consteval info from() const noexcept;
```

4 *Returns:* *from_*.

```
    consteval source_location where() const noexcept;
```

5 *Returns:* *where_*.

§ 4.3 [meta.reflection.operators]

Replace the error handling in this subclause:

- ```
consteval operators operator_of(info r);
```
- 2 ~~Constant When~~ **Throws:** `meta::exception unless` `r` represents an operator function or operator function template.
- 3 **Returns:** The value of the enumerator from `operators` whose corresponding *operator-function-id* is the unqualified name of the entity represented by `r`.
- ```
consteval string_view symbol_of(operators op);
consteval u8string_view u8symbol_of(operators op);
```
- 4 ~~Constant When~~ **Throws:** `meta::exception unless the` ~~The~~ value of `op` corresponds to one of the enumerators in `operators`.
- 5 **Returns:** `string_view` or `u8string_view` containing the characters of the operator symbol name corresponding to `op`, respectively encoded with the ordinary literal encoding or with UTF-8.

§ 4.4 [meta.reflection.names]

Replace the error handling in this subclause:

- ```
consteval string_view identifier_of(info r);
consteval u8string_view u8identifier_of(info r);
```
- 1 Let *E* be UTF-8 if returning a `u8string_view`, and otherwise the ordinary literal encoding.
- 2 ~~Constant When~~ **Throws:** `meta::exception unless` `has_identifier(r)` is `true` and the identifier that would be returned (see below) is representable by *E*.

## § 4.5 [meta.reflection.queries]

Replace the error handling in this subclause:

- ```
consteval info type_of(info r);
```
- 1 ~~Constant When~~ **Throws:** `meta::exception unless` `r` represents a value, object, variable, function that is not a constructor or destructor, enumerator, non-static data member, bit-field, direct base class relationship, or data member description.

```
constexpr info object_of(info r);
```

- 2 ~~Constant When~~ **Throws**: `meta::exception unless` `r` represents either an object with static storage duration ([basic.stc.general]), or a variable associated with, or referring to, such an object.

```
constexpr info value_of(info r);
```

- 3 ~~Constant When~~ **Throws**: `meta::exception unless` `r` is a reflection representing
- (3.1) — a value,
 - (3.2) — an enumerator, or
 - (3.3) — an object or variable `X` such that the lifetime of `X` has not ended, the type of `X` is a structural type, and either `X` is usable in constant expressions from some point in the evaluation context or the lifetime of `X` began in the manifestly constant-evaluated expression currently under evaluation ([expr.const]), ([temp.type]).

```
constexpr info parent_of(info r);
```

- 4 ~~Constant When~~ **Throws**: `meta::exception unless` `r` represents a variable, structured binding, function, enumerator, class, class member, bit-field, template, namespace or namespace alias (other than `::`), type alias, or direct base class relationship.

```
constexpr info template_of(info r);  
constexpr vector<info> template_arguments_of(info r);
```

- 5 ~~Constant When~~ **Throws**: `meta::exception unless` `has_template_arguments(r)` is `true`.

§ 4.6 [meta.reflection.member.queries]

Replace the error handling in this subclause:

```
constexpr vector<info> members_of(info r);
```

- 1 ~~Constant When~~ **Throws**: `meta::exception unless` `r` is a reflection representing either a class type that is complete from some point in the evaluation context or a namespace.

```
constexpr vector<info> bases_of(info type);
```

- 2 ~~Constant When~~ Throws: `meta::exception unless` `dealias(type)` is a reflection representing a complete class type.

```
constexpr vector<info> static_data_members_of(info type);
```

- 3 ~~Constant When~~ Throws: `meta::exception unless` `dealias(type)` represents a complete class type.

```
constexpr vector<info> nonstatic_data_members_of(info type);
```

- 4 ~~Constant When~~ Throws: `meta::exception unless` `dealias(type)` represents a complete class type.

```
constexpr vector<info> enumerators_of(info type_enum);
```

- 5 ~~Constant When~~ Throws: `meta::exception unless` `dealias(type_enum)` represents an enumeration type and `has_complete_definition(dealias(type_enum))` is `true`.

```
constexpr vector<info> get_public_members(info type);
```

- 6 ~~Constant When~~ Throws: `meta::exception unless` `dealias(type)` represents a complete class type.

```
constexpr vector<info> get_public_bases(info type);
```

- 7 ~~Constant When~~ Throws: `meta::exception unless` `dealias(type)` represents a complete class type.

```
constexpr vector<info> get_public_static_data_members(info type);
```

- 8 ~~Constant When~~ Throws: `meta::exception unless` `dealias(type)` represents a complete class type.

```
constexpr vector<info> get_public_nonstatic_data_members(info type);
```

- 9 ~~Constant When~~ Throws: `meta::exception unless` `dealias(type)` represents a complete class type.

§ 4.7 [meta.reflection.layout]

Replace the error handling in this subclause:

```
constexpr member_offset offset_of(info r);
```

- 1 ~~Constant When~~ **Throws:** `meta::exception unless` `r` represents a non-static data member, unnamed bit-field, or direct base class relationship other than a virtual base class of an abstract class.

```
constexpr size_t size_of(info r);
```

- 2 ~~Constant When~~ **Throws:** `meta::exception unless` `dealias(r)` is a reflection of a type, object, value, variable of non-reference type, non-static data member, direct base class relationship, or data member description. If `dealias(r)` represents a type T , there is a point within the evaluation context from which T is not incomplete.

```
constexpr size_t alignment_of(info r);
```

- 3 ~~Constant When~~ **Throws:** `meta::exception unless` `dealias(r)` is a reflection representing a type, object, variable, non-static data member that is not a bit-field, direct base class relationship, or data member description. If `dealias(r)` represents a type T , there is a point within the evaluation context from which T is not incomplete.

```
constexpr size_t bit_size_of(info r);
```

- 4 ~~Constant When~~ **Throws:** `meta::exception unless` `dealias(r)` is a reflection of a type, object, value, variable of non-reference type, non-static data member, unnamed bit-field, direct base class relationship, or data member description. If `dealias(r)` represents a type T , there is a point within the evaluation context from which T is not incomplete.

§ 4.8 [meta.reflection.extract]

Replace the error handling in this subclause:

```
template <class T>
constexpr T extract-ref(info r); // exposition only
```

- 1 [Note 1: T is a reference type. — end note]

- 2 ~~Constant When~~ **Throws:** `meta::exception unless` `r` represents a variable or object of type U that is usable in constant expressions from some point in the evaluation context and

```
is_convertible_v<remove_reference_t<U>(*)[], remove_reference_t<T>(*)[]> is
true.
```

```
template <class T>
    consteval T extract-member-or-function(info r); // exposition only
```

3 ~~Constant When~~ **Throws:** meta::exception unless the following conditions are met:

- (3.1) — If r represents a non-static data member of a class C with type X, ~~then when~~ T is X C::*, and r does not represent a bit-field.
- (3.2) — Otherwise, if r represents an implicit object member function of class C with type F or F noexcept, ~~then when~~ T is F C::*.
- (3.3) — Otherwise, r represents a function, static member function, or explicit object member function of function type F or F noexcept, ~~then when~~ and T is F*.

```
template <class T>
    consteval T extract-val(info r); // exposition only
```

4 Let U be the type of the value that r represents.

5 ~~Constant When~~ **Throws:** meta::exception unless the following conditions are met:

- (5.1) — U is a pointer type, T and U are similar types ([conv.qual]), and is_convertible_v<U, T> is true,
- (5.2) — U is not a pointer type and the cv-unqualified types of T and U are the same, or
- (5.3) — U is a closure type, T is a function pointer type, and the value that r represents is convertible to T.

§ 4.9 [meta.reflection.substitute]

Replace the error handling in this subclause:

```
template <reflection_range R = initializer_list<info>>
    consteval bool can_substitute(info templ, R&& arguments);
```

1 ~~Constant When~~ **Throws:** meta::exception unless templ represents a template and every reflection in arguments represents a construct usable as a template argument ([temp.arg]).

```
template <reflection_range R = initializer_list<info>>
    consteval info substitute(info templ, R&& arguments);
```

2 ~~Constant When~~ **Throws:** `meta::exception unless` `can_substitute(templ, arguments)` is `true`.

§ 4.10 [meta.reflection.result]

Replace the error handling in this subclause:

```
template <typename T>
constexpr info reflect_value(const T& expr);
```

1 *Mandates:* T is a structural type that is neither a reference type nor an array type.

2 ~~Constant When~~ **Throws:** `meta::exception unless any` ~~Any~~ value computed by `expr` having pointer type, or every subobject of the value computed by `expr` having pointer or reference type, is the address of or refers to an object or function that

- (2.1) — is a permitted result of a constant expression ([`expr.const`]),
- (2.2) — is not a temporary object ([`class.temporary`]),
- (2.3) — is not a string literal object ([`lex.string`]),
- (2.4) — is not the result of a `typeid` expression ([`expr.typeid`]), and
- (2.5) — is not an object associated with a predefined `__func__` variable ([`dcl.fct.def.general`]).

```
template <typename T>
constexpr info reflect_object(T& expr);
```

3 *Mandates:* T is not a function type.

4 ~~Constant When~~ **Throws:** `meta::exception unless` `expr` designates an object or function that

- (4.1) — is a permitted result of a constant expression ([`expr.const`]),
- (4.2) — is not a temporary object ([`class.temporary`]),
- (4.3) — is not a string literal object ([`lex.string`]),
- (4.4) — is not the result of a `typeid` expression ([`expr.typeid`]), and
- (4.5) — is not an object associated with a predefined `__func__` variable ([`dcl.fct.def.general`]).

§ 4.11 [meta.reflection.define.aggregate]

Replace the error handling in this subclause:

```
constexpr info data_member_spec(info type,  
                                data_member_options options);
```

1 **Constant When Throws:** `meta::exception` unless the following conditions are met:

- (1.1) — `dealias(type)` represents a type `cv T` where `T` is either an object type or a reference type;
- (1.2) — if `options.name` contains a value, then:
 - (1.2.1) — `holds_alternative<u8string>(options.name->contents)` is `true` and `get<u8string>(options.name->contents)` contains a valid identifier when interpreted with UTF-8, or
 - (1.2.2) — `holds_alternative<string>(options.name->contents)` is `true` and `get<string>(options.name->contents)` contains a valid identifier when interpreted with the ordinary literal encoding;
- (1.3) — otherwise, if `options.name` does not contain a value, then `options.bit_width` contains a value;
- (1.4) — if `options.alignment` contains a value, it is an alignment value (`[basic.align]`) not less than `alignment_of(type)`; and
- (1.5) — if `options.bit_width` contains a value `V`, then
 - (1.5.1) — `is_integral_type(type) || is_enumeration_type(type)` is `true`,
 - (1.5.2) — `options.alignment` does not contain a value,
 - (1.5.3) — `options.no_unique_address` is `false`, and
 - (1.5.4) — if `V` equals `0` then `options.name` does not contain a value.

§ 5 References

[LWG4087] Victor Zverovich. Standard exception messages have unspecified encoding.

<https://wg21.link/lwg4087>

[P2996R8] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, Dan Katz. 2024-12-17. Reflection for C++26.

<https://wg21.link/p2996r8>

[P3068R6] Hana Dusíková. 2024-11-19. Allowing exception throwing in constant-evaluation.

<https://wg21.link/p3068r6>